

Artificial Intelligence

Supervised Learning

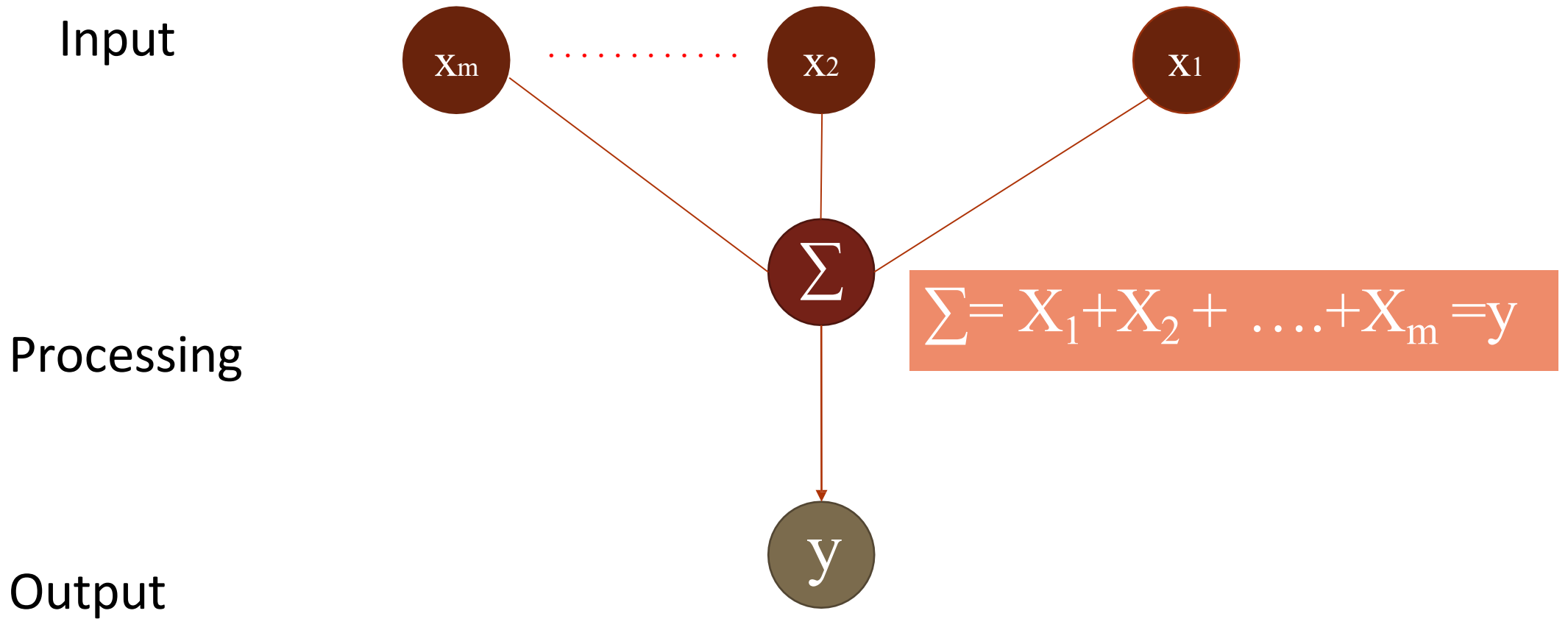


Artificial Neural Network

- Computational models **inspired by the human brain:**
 - **Massively parallel, distributed system**, made up of simple processing units (neurons)
 - **Synaptic connection strengths** among neurons are used to store the acquired knowledge.
 - Knowledge is acquired by the network from its environment through a **learning process**



ANN





ANN

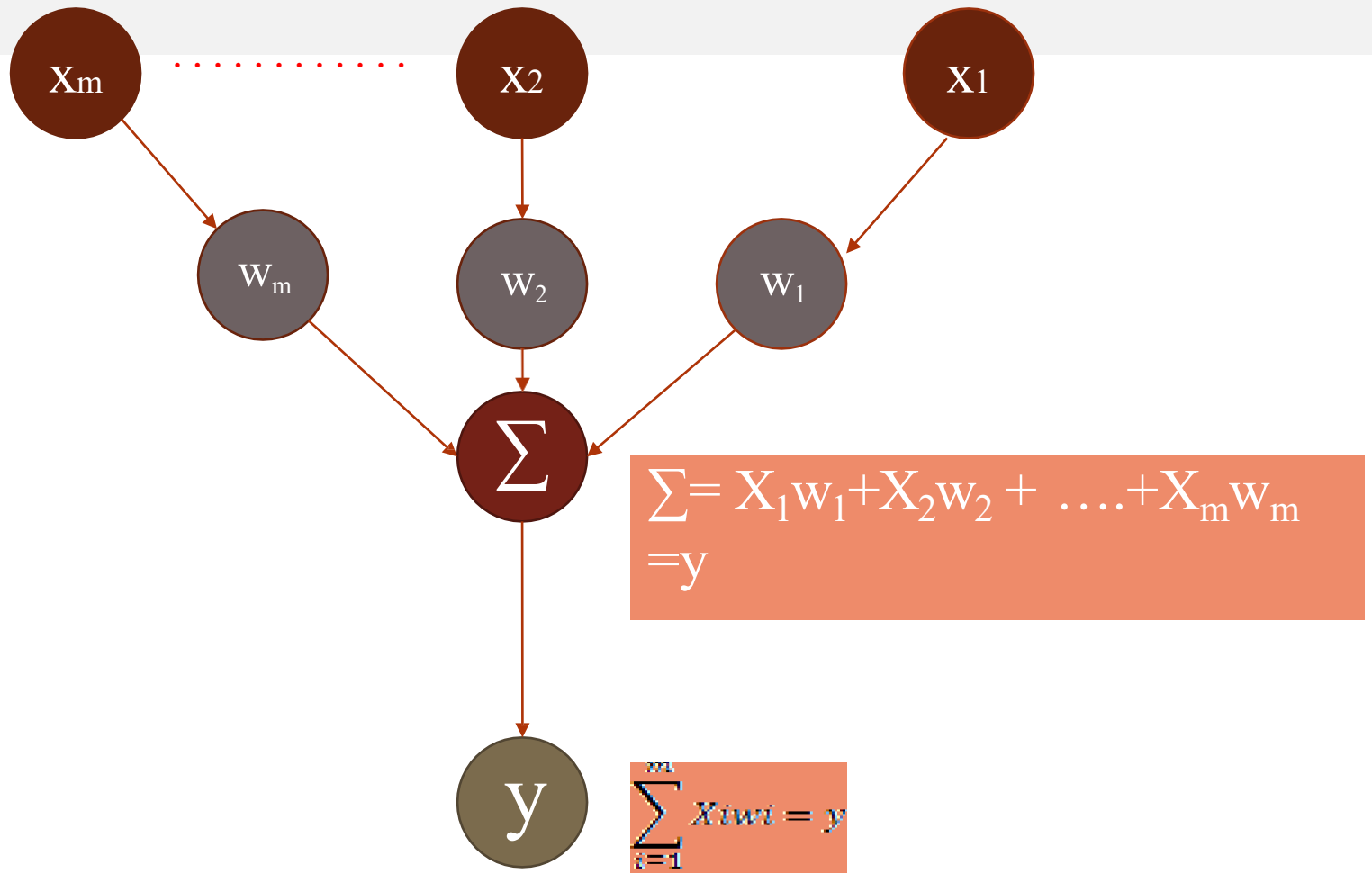
Input

Weights

Processing

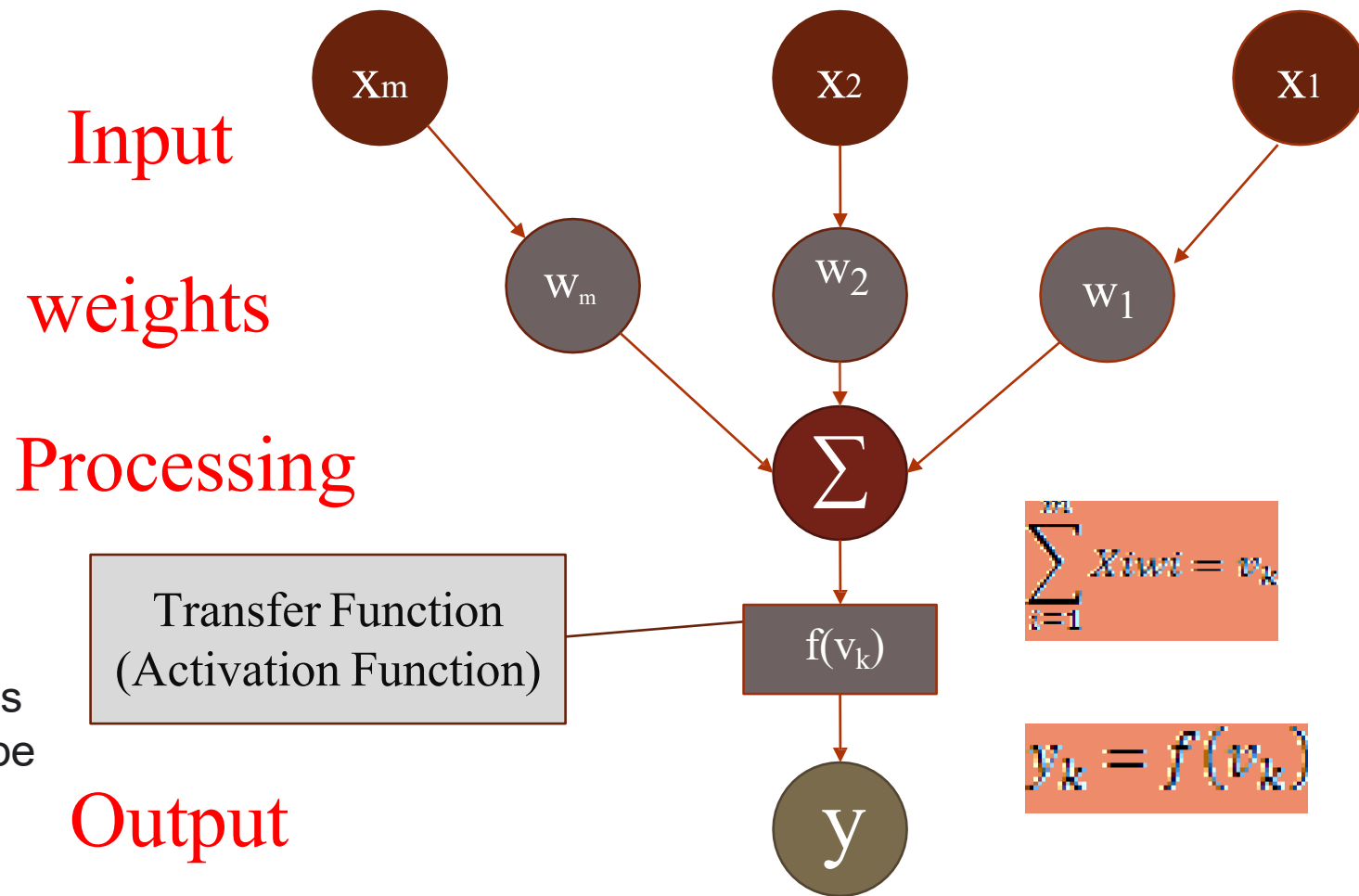
Output

Not all inputs are equal





ANN



The purpose of the **activation function** is to **add non-linearity to the neural** networks. It decides whether a neuron should be activated or not

Output

Common Activation Functions

1. Binary Step Function

(a “threshold” or “Heaviside” function)

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

2. Bipolar Step Function

$$f(x) = \begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

3. Binary Sigmoid Function

(Logistic Sigmoid)

$$f(x) = \frac{1}{1 + \exp(-\sigma x)}; f'(x) = \sigma f(x)[1 - f(x)]$$

3. Bipolar Sigmoid

$$g(x) = \frac{1 - \exp(-\sigma x)}{1 + \exp(-\sigma x)}; g'(x) = \frac{\sigma}{2} [1 + g(x)][1 - g(x)]$$

4. Hyperbolic - Tangent

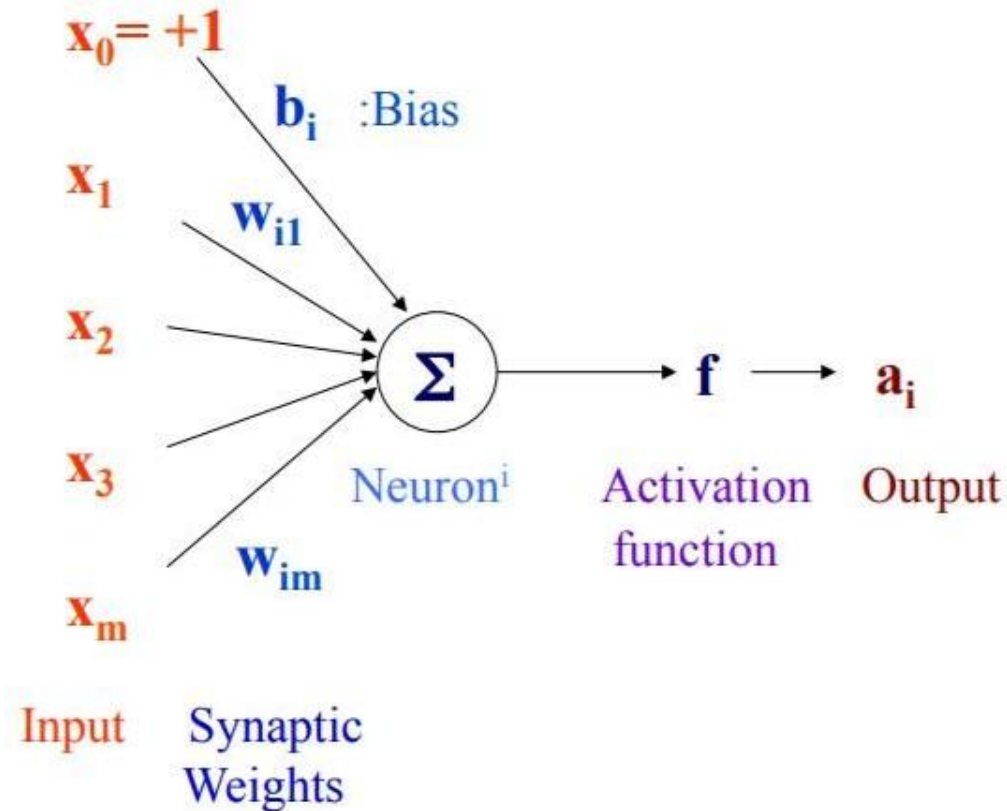
$$h(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}; h'(x) = [1 + h(x)][1 - h(x)]$$



Artificial Neural Network

Weight determines the connection between the two neurons.

Each connection of neurons has its own weight, and those are the only values that will be modified during the learning process.





Artificial Neural Network

$$a_i = f(n_i) = f\left(\sum_{j=1}^n w_{ij}x_j + b_i\right)$$

An artificial neuron:

- computes the **weighted sum of its input** (called its **net input**)
- adds its bias
- passes this value through an activation function

We say that the neuron “**fires**” (i.e. becomes active) if its output is above zero.



Artificial Neural Network

Bias can be incorporated as another weight clamped to a fixed input of +1.0

This extra free variable (bias) makes the neuron more powerful.

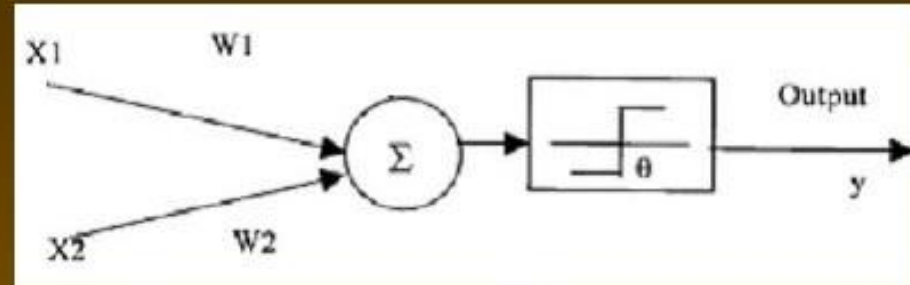
$$a_i = f(n_i) = f\left(\sum_{j=0}^n w_{ij} x_j\right) = f(\mathbf{w}_i \cdot \mathbf{x}_j)$$

Example: Working of a Typical ANN

Artificial Neuron Model

Implementation of AND function

Let $W_1 = W_2 = 1$



x_1	x_2	$x_1 w_1 + x_2 w_2$	Y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

If we make $\theta = 2$ (or any value >1 but ≤ 2), we will get correct results with a unit step activation function

Unit step activation

It takes an input value and returns a binary output based on a threshold.

Artificial Neuron Model

If we place the 4 points in a two coordinate system (X_1 and X_2), we have drawn a line from $(2, 0)$ to $(0, 2)$ in the resulting plane

Any new data falling on the left side of the line will give an output of zero and the data on the right side of the line will be classified as one

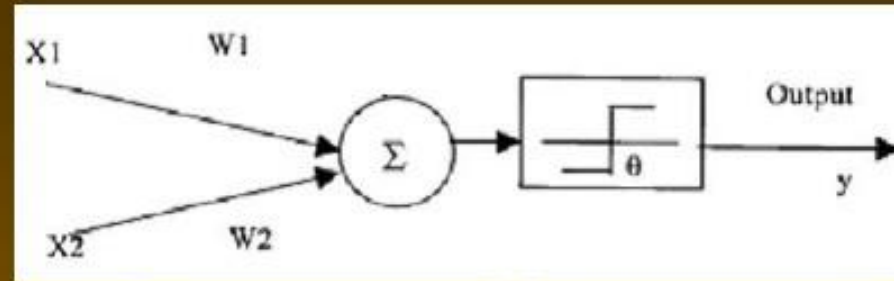


Example: Working of a Typical ANN

Artificial Neuron Model

Implementation of OR function

Let $W_1 = W_2 = 1$



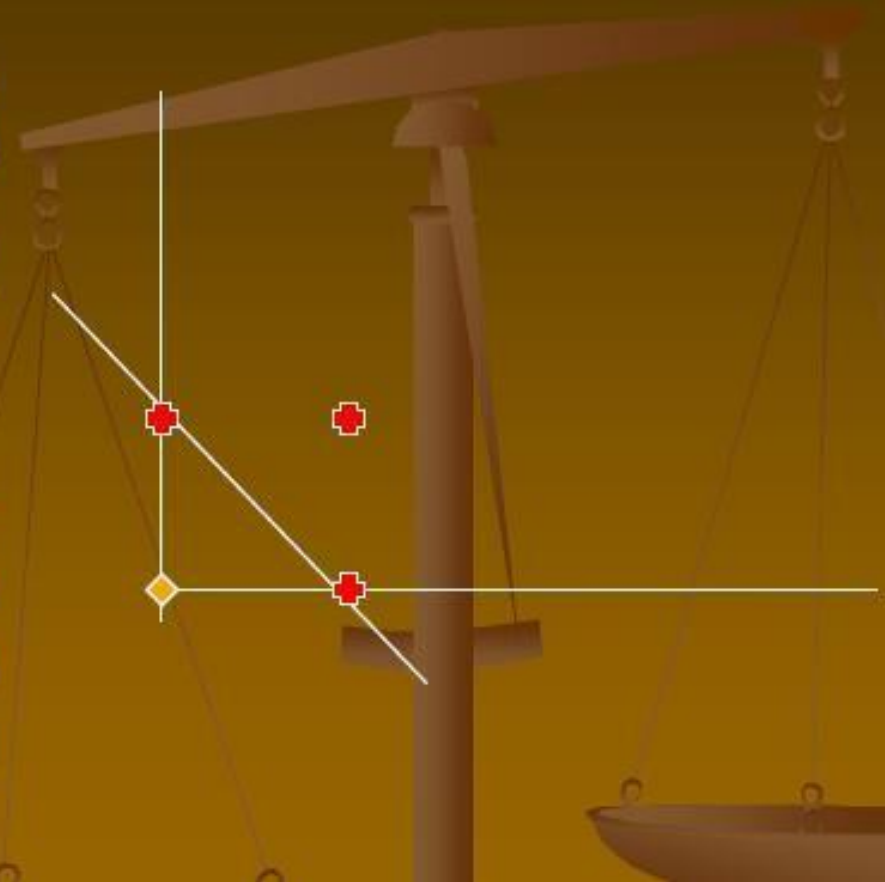
X_1	X_2	$X_1W_1 + X_2W_2$	Y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

If we make $\theta = 1$ (or any value >0 but ≤ 1), we will get correct results with a unit step activation function

Artificial Neuron Model

If we place the 4 points in a two coordinate system (X_1 and X_2), we have drawn a line from $(1, 0)$ to $(0, 1)$ in the resulting plane

Any new data falling on the left side of the line will give an output of zero and the data on the right side of the line will be classified as one





Learning:

Continuous process of:

Trial:

Processing an input to produce an output (In terms of ANN: Compute the output function of a given input)

Evaluate:

Evaluating this output by comparing the actual output with the expected output.

Adjust:

Adjust the weights.



Representing Boolean Function

Simple example: AND

$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$

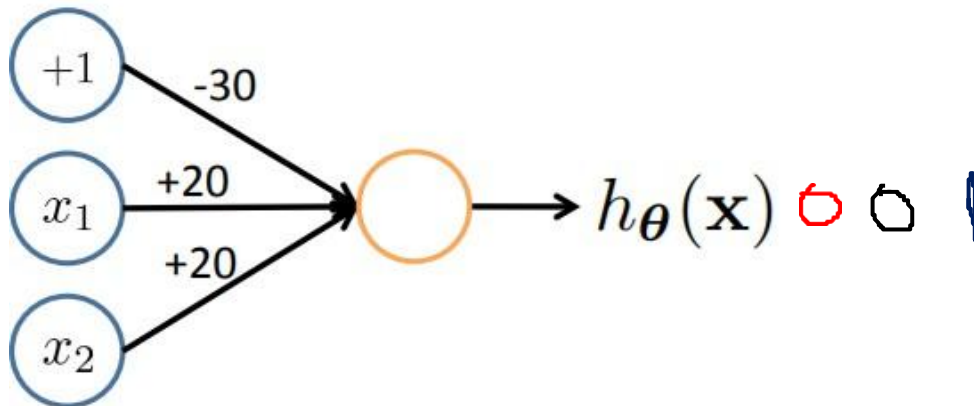
x_1	x_2	$h_{\Theta}(\mathbf{x})$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

Representing Boolean Function

Simple example: AND

$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$



$$h_{\theta}(\mathbf{x}) = g(-30 + 20x_1 + 20x_2)$$

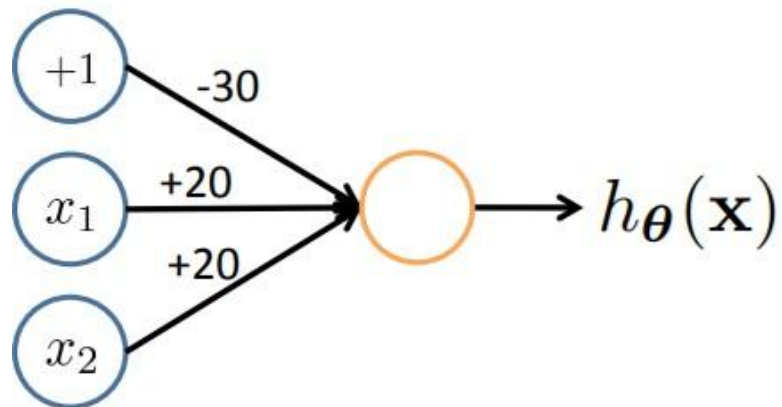
x_1	x_2	$h_{\theta}(\mathbf{x})$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

Representing Boolean Function

Simple example: AND

$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$

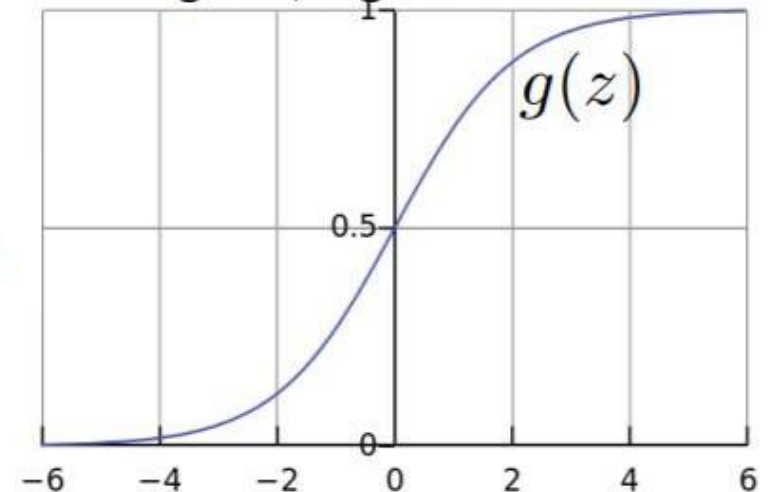


$$h_{\theta}(\mathbf{x}) = g(-30 + 20x_1 + 20x_2)$$

x_1	x_2	$h_{\theta}(\mathbf{x})$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

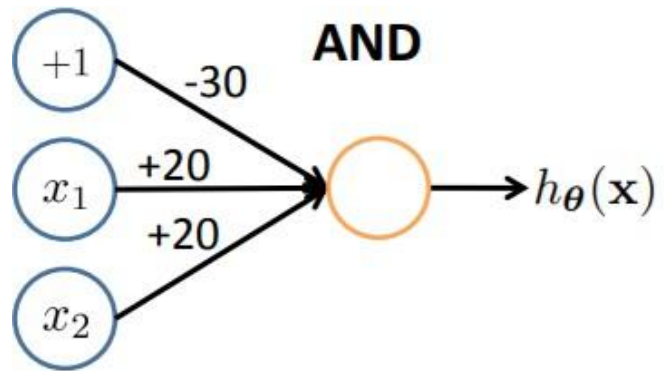
$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Logistic / Sigmoid Function

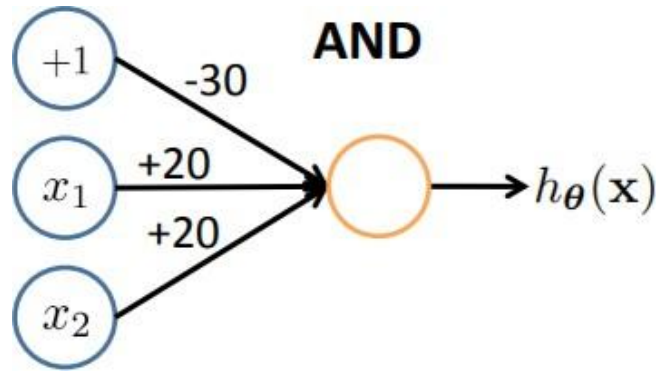




Representing Boolean Function



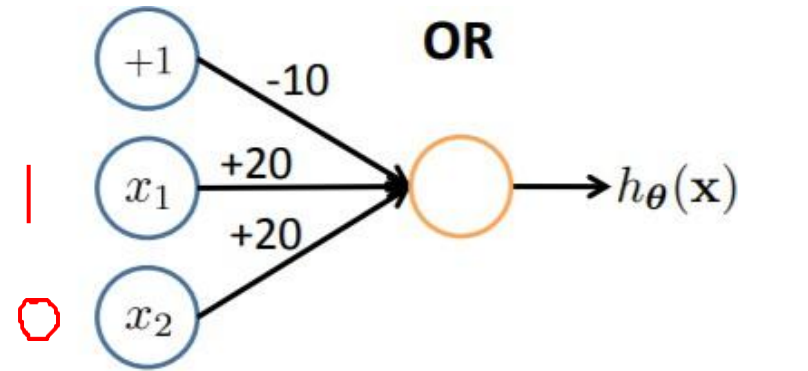
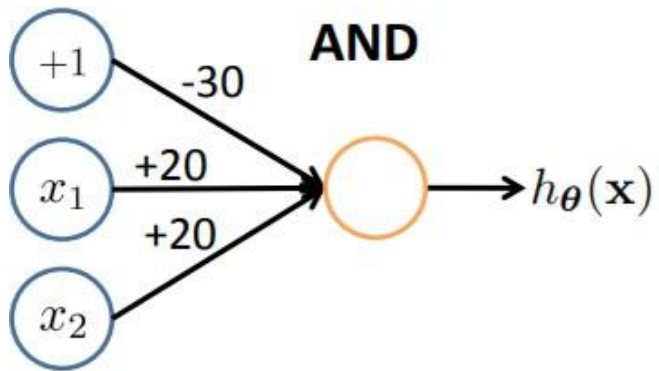
Representing Boolean Function



What changes will you make to convert this network into an OR function?

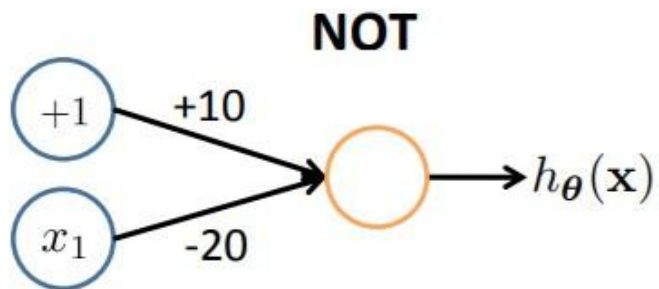
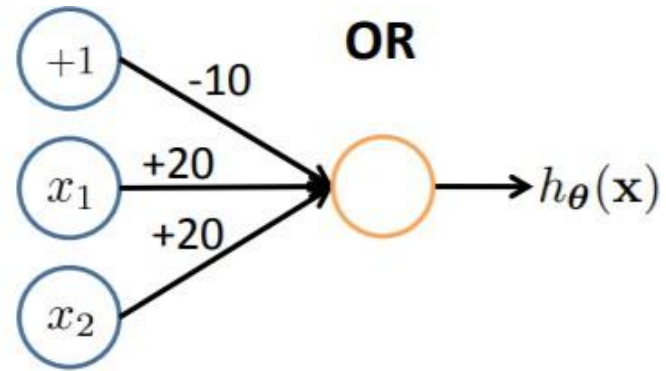
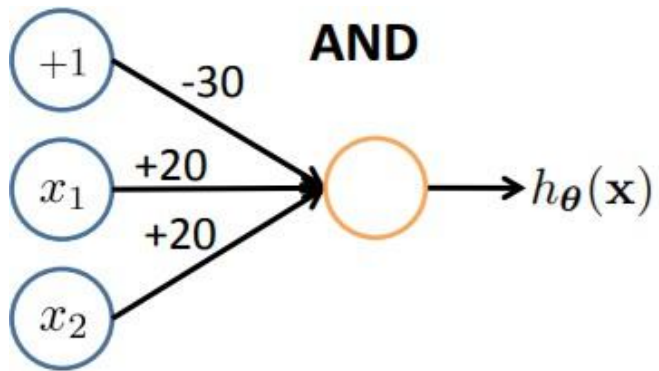


Representing Boolean Function

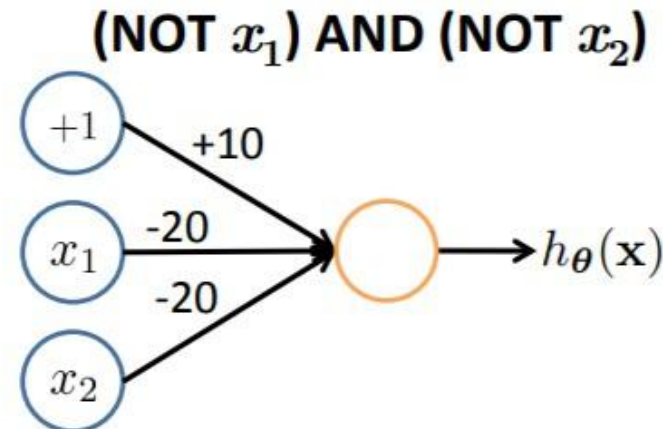
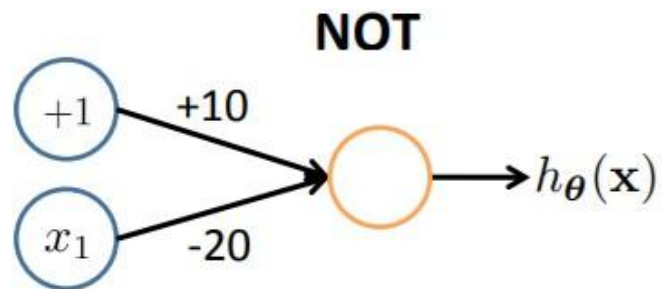
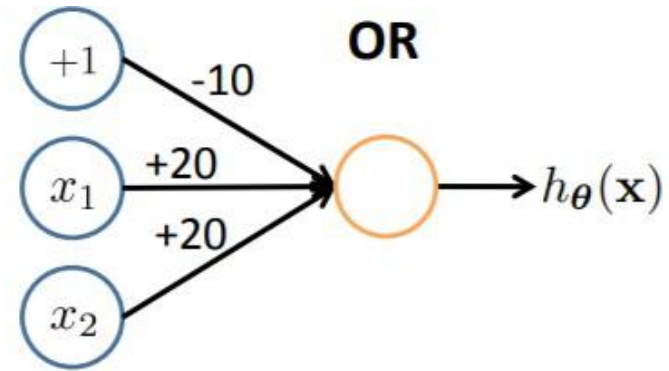
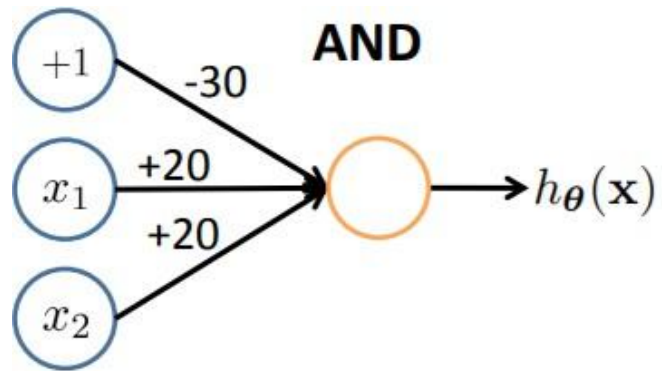




Representing Boolean Function



Representing Boolean Function



Show the mathematical working of Artificial Neural Network by taking the case in figure below. First two columns are the input values for X1 and X2 and the third column is the desired output.

0	0	0
0	1	0
1	0	1
1	1	1

Learning rate = 0.2

Threshold = 0.5

Actual output = $W_1X_1 + W_2X_2$

Next weight adjustment = $\underline{W_n} + \underline{\Delta W_n}$

Change in weight = $\underline{\Delta W_n} = \text{learning rate} * (\text{desired output} - \text{output}) * \underline{X_n}$

Show complete iterations for acquiring the desired output?

x1	x2	w1	w2	d	y	Δw_1	Δw_2
----	----	----	----	---	---	--------------	--------------

$$y = x_1w_1 + x_2w_2 \quad \Delta w_1 = \eta(d - a) \cdot x_1 \quad w_{n1} = w_0 + \Delta w_n \quad \eta = 0.2 \quad \theta = 0.5$$

$$y = x_1 w_1 + x_2 w_2 \quad \Delta w_i = \eta (d - a) \cdot x_i \quad w_N = w_0 + \Delta w_N \quad \eta = 0.2$$

$$\theta = 0.5$$

R.W

x_1	x_2	w_1	w_2	d	y	Δw_1	Δw_2	
0	0	1	1	0	0	0	0.2	$0.2(0-1) = -0.2$
0	1	1	1	0	1	0	-0.2	$1.8 > 0.5 = 1$
1	0	1	0.8	1	1	0	0	
1	1	1	0.8	1	1	0	0	
0	0	1	0.8	0	0	0	0	$0.8 > 0.5$
0	1	1	0.8	0	1	0	-0.2	$0.2(0-1) = -0.2$
1	0	1	0.6	1	1	0	0	$1 + 0.6 = 1.6 > 0.5 = 1$
1	1	1	0.6	1	1	0	0	
0	0	1	0.6	0	0	0	0	$0.6 > 0.5 = 1$
0	1	1	0.6	0	1	0	-0.2	
1	0	1	0.4	1	1	0	0	$1 > 0.5 = 1$
1	1	1	0.4	1	1	0	0	$1 + 0.4 = 1.4 > 0.5 = 1$
0	0	1	0.4	0	0	0	0	$0.4 < 0.5 = 0$
0	1	1	0.4	0	0	0	0	
1	0	1	0.4	1	1	0	0	$1 + .4 = 1.4 > 0.5 = 1$
1	1	1	0.4	1	1	0	0	

$$(1-0) \quad y = 1 \times 1 + 0 \times 0.4 = 1 + 0 = 1 > 0.5 = 1$$

$$w_1 = 1$$

$$w_2 = 0.4$$

$$(0-1) \quad y = 0 \times 1 + 1 \times 0.4 = 0.4 < 0.5 = 0$$

$$(0-0) \quad y = 0 \times 1 + 0 \times 0.4 = 0$$

$$(1-1) \quad y = 1 \times 1 + 1 \times 0.4 = 1 + 0.4 = 1.4 > 0.5 = 1$$